

תכנון:

ארכיטקטורה של CC בIoT:

בתוך רכיב הIoT יש 2 מערכות הפעלה:

1. מערכת הפעלה לינוקסית רגילה – נחשבת חלק מהעולם הלא מאובטח.
2. OP-TEE מערכת הפעלה מיוחדת, רזה ומאובטחת (open source) שרצה בתוך העולם המאובטח ומנהלת את enclaves שרצים ברכיב.

TrustedZone: רכיב ארכיטקטוני שמשמש לפיצול ל2 העולם (מאובטח / לא) של ה:

1. מעבד של ARM ברכיב הIoT
2. Bus
3. זיכרון

דגש – פה הם קוראים ל enclave בכינוי (Trusted Application) TA. ההבדל: בעולם של intel הבידוד הוא בעצם זיכרון וירטואלי בתוך הפרוסס של enclaves שרץ ב user space ומנוהל על ידי הקרנל (אך לקרנל אין גישה לתוך הזיכרון).
בTAs של ARM - TA לא רץ בכלל כפרוסס של לינוקס אלא רץ במערכת הפעלה המיועדת של OP-TEE.

CA Client Application - לכל TA ברכיב הIoT יש אפליקציה רגילה שחיה בעולם הלא מאובטח במערכת הפעלה הרגילה של לינוקס והיא מנהלת ומפעילה את TA ונהנית מביצוע פעולות באופן מאובטח.

נשים לב – בintel ראינו שכדי לקרוא לפונקציה מתוך אפליקציה רגילה כך שתרוץ בעולם היינו צריכים להשתמש בעצמנו בcall. בARM זה נקרא TEEC_InvokeCommand.

בנוסף, כדי שהעולם של intel יוכל לבצע קריאה לפונקציה שמחייבת יציאה מהאזור המאובטח בintel השתמשנו בsyscall (למשל כדי להדפיס למסך). בARM זה נקרא RPC (Remote Procedure Call).

פקודת syscall לביצוע content switch בין הקרנל הלא מאובטח למאובטח – SMC (Secure Monitor Call).

כאשר כותבים TA (קוד C מוגבל) משתמשים בAPI ש OP-TEE מספקת. לכן הפרויקט תמיד יכול ל2 תיקיות:

1. Host – עבור הCAs שרצים בלינוקס.
2. Ta – עבור הלוגיקות המאובטחות.

זרימת עלייה של המערכת:

1. שכבת האמולציה – qemu מדמה את המעבד והזיכרון של ARM.
2. שכבת הfirmware – הקוד הראשון שרץ. ל-ARM יש מימוש bootloader מאובטח בשם TF-A (Trusted Firmware).
3. העולם המאובטח – OP-TEE OS
 - מערכת ההפעלה והקרנל המאובטח.
 - TAs.
4. העולם הרגיל –
 - לינוקס קרנל רגיל מכיל דרייבר עבור OP-TEE.
 - Client Daemon – תוכנת רקע שמסייעת לדרייבר לתקשר עם העולם המאובטח.
 - Cas.

דגש – רק הCAs מבצעים פעולות רשת!

על מה מתבססים:

QEMU – אמולטור חומרה נפוץ. הוא מסוגל לדמות לוח ARM שתומך בTrustZone על מנת לדמות מכשיר IoT.

ישנו פרויקט OP-TEE QEMU Manifest ובו קיימים:

- תשתית הbuild – מערכת שיוצרת מערכת הפעלה שמקונפגת לעבוד עם OP-TEE.
- אפליקציות לדוגמה.
- API לפעולות קריפטוגרפיות.

גיט: <https://github.com/OP-TEE/manifest>

הארכיטקטורה במידול שלנו (מציאותי):

כל לוח PCB של חיישן מורכב מ:

- אלמנט החישה – הרכיב שדוגם את הנתונים מהסביבה.
- Secure Element – שבב קטן, מוטמע ביחד עם רכיב החישה.

ב-Secure Element יש את היכולת:

- להחזיק סוד צרוב ולספק יכולת הזדהות חד ערכית של כל לוח הPCB.
- הוא מחזיק crypto engine
- Secure Key Storage – לאחסון מפתחות, סרטיפיקטים וסיסמאות
- RAM מוגבל

אין לו:

- MCU כללי (למטרות כלליות שאינן קריפטוגרפיה מובנית).
- יכולת להריץ OS
- MMU

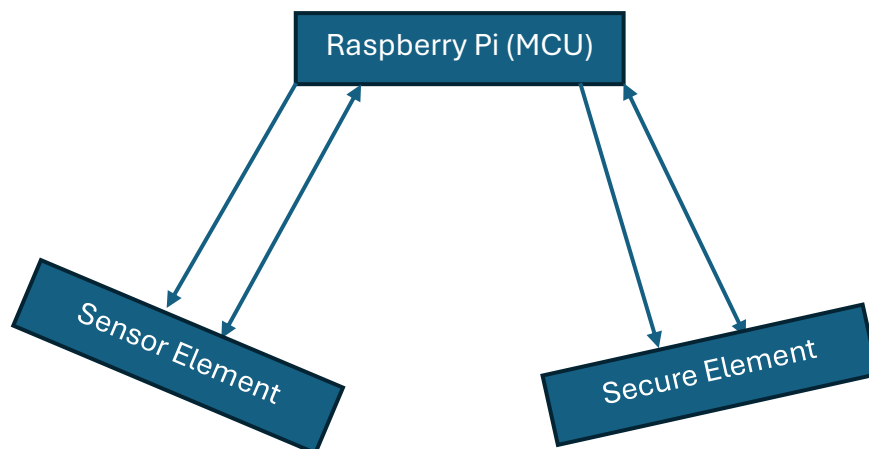
דוגמה: ATECC608A



הוא slave device בדיוק כמו אלמנט החישה. כלומר הוא לא יכול ליזום בקשות להזרמת מידע או ביצוע פעולות, הוא עושה מה שרכיב ה - MCU או CPU חכם יודע לתזמן ולבקש ממנו.

ה edge device הוא רכיב כמו Raspberry pi בעל זיכרון יותר גדול וMCU (או CPU) לשימוש כללי ולכן הוא יכול להחזיק ארכיטקטורת TEE מותאמת לOS בכללי. לוגיקת ניהול הPCB, קבלת המידע הממנו, הצפנה ושליחת הנתונים שהתקבלו לשרת תתבצע בTE ייעודי בתוך מערכת ההפעלה של OP-TEE בלבד.

ה device מתפקד כmaster ללוח הPCB – הוא יכול ליזום בקשת פעולות קריפטוגרפיות והזרמת נתונים בפרוטוקול I2C (דרך חוטים של SDL להעברת הdata וSCL – שעון לתזמון פעולות ומתן (קבלה) פקודות לslaves (מהmaster)).



לכן בפועל אמנם לsecure element יש יכולת להצפין את הנתונים שמגיעים מהחיישן אבל אין לנתונים כל יכולת להגיע לsecure element מהחיישן באופן ישיר. הדרך היחידה שבה ניתן להצפין את הנתונים מהחיישן באמצעות הsecure element היא להעביר את המידע מהחיישן לMCU ואז MCU מבקש להצפין את הנתונים – אבל זה מיותר כי בארכיטקטורה שלנו MCU הוא ARM TEE שלנו ולכן הוא יכול בעצמו להצפין את הנתונים ולשלוח אותם לשרת. לכן מטרת הsecure element היא להעניק זהות קריפטוגרפית לsensor element באמצעות מיקומו על אותו לוח PCB יחד עם החיישן – אין בניהם יכולת העברת נתונים ואין יכולת לנתונים הנאספים מהחיישן לצאת מהלוח באופן מוצפן מראש.

לכן כל עוד לחיישן עצמו אין באופן אינהרנטי יכולת להצפין בעצמו את הנתונים (עם MCU ומשאבים יותר חזקים) אין פתרון קריפטוגרפי להתמודד עם בעיות אבטחה על הSCL / SDL links. נהוג לנקוט באמצעים ברמה העיצובית-חומרית כדי למנוע מתוקף לבצע probing על החוטים למשל:

- למקם את החוטים האלה באופן מוטמע בתוך לוח PCB גדול שכולל את כל הישויות ולחווט אותם בשכבות פנימיות של הלוח על מנת שלא יהיו חשופים או ימוקמו בשכבות החיצוניות של הלוח.
- להימנע ממיקום test points ופינים חשופים שקשורים לחוטים האלו כדי למנוע הימצאות נקודות גישה probe גם לתוקף.
- מיקום הרכיבים בצפיפות יחסית זה לזה כדי להקטין את אורכי החוטים ולמזער את משטח הגישוש עם אמצעי probe.

Project Description:

Assets:

- Raw Sensor Data (at rest and in transit)
- Sensor Module
- Device-Server session key
- Device firmware and security critical application logic

Defense Metrics:

- Raw Sensor Data Integrity
- Raw Sensor Data Confidentiality
- Cryptographic Keys Confidentiality
- Firmware and Device Code Files Integrity
- Sensor Module (Sensor and Secure elements) integrity

Trusted Computing Base:

- Sensor internal logic
- Secure Element logic and embedded secret

- Sensor-to-Device and SecureElement-to-Device communication wires
- OP-TEE OS
- Trusted Application (TA)
- Server

Outside the Trusted Computing Base:

- Edge Device Normal World (Rich OS, Linux)
- User applications running on the Edge Device
- Communication network
- End user

Attacker Profile:

External attacker with no trusted credentials and no access to the Trusted Computing Base.

Attacker Capabilities:

- Physical access to the Sensor Module and Edge Device.
- Full control over the communication network (Passive and active network attacks).
- Ability to execute arbitrary software on the Edge Device Normal World.
- Ability to inject malicious firmware into the Normal World.
- Replay, spoofing and message modification

Assumptions:

- Cannot compromise secure element (internal logic, storage and cryptographic keys, embedded secret).
- Cannot corrupt sensor internal logic.
- Cannot compromise OP-TEE or the TA.
- Cannot forge a valid attestation report.
- Cannot compromise the backend server.

Threat Scenarios:

1. Sensor Module Replacement Attack

The attacker is capable of physically replacing the complete board of the legitimate sensor module with a malicious one compromising the integrity of the raw sensor data.

2. Edge Device Software Compromise

The attacker can compromise the Device through software attacks and gaining control over its operating system to:

- modify the device's application logic and firmware.
- steal sensitive system information such as cryptographic keys or session keys established with the server.
- impersonate the Device when communicating with the server.

This attack compromises the integrity and confidentiality of the raw sensor data, the integrity of the firmware and device code, and the confidentiality of cryptographic keys.

3. Man-in-the-Middle (MITM) Attack

The attacker can perform both passive and active Man-in-the-Middle attacks on the communication channel between the Edge Device and the server, allowing the attacker to eavesdrop on, modify, inject, or replay transmitted messages. This compromises the confidentiality and integrity of the raw sensor data.

Project Goals:

- Ensure that only authenticated sensor modules can provide the Sensor Identity Secret to the Edge Device using an embedded secure element.
- Ensure integrity of the sensor data from acquisition until arrival at the server.
- Ensure confidentiality of sensor data while transmitted over untrusted links.
- Prevent unauthorized disclosure of cryptographic material.
- Ensure that only an untampered Edge Device may communicate with the server.
- Prevent execution of unauthorized code inside the trusted execution environment.

Mitigations:

Threat	Mitigation
1. Sensor Module Replacement Attack	The sensor module must authenticate itself to the Edge Device using a Challenge–Response protocol based on the secure element’s embedded secret, preventing unauthorized sensors from impersonating legitimate devices.
2. Edge Device Software Compromise	Deploy ARM TrustZone on the Edge Device. Security-critical code responsible for sensor authentication, sensor data decryption, cryptographic key management, and message preparation is executed exclusively inside a protected Trusted Application (TA).

2. Device Controller Impersonation	Perform Remote Attestation, where the Edge Device acts as the Prover and the Server acts as the Verifier, allowing the server to verify the integrity of the Device before establishing trust.
3. Man-in-the-Middle (MITM) Attack	Protect communication between the Edge Device and the Server using authenticated encryption and MAC (to provide both confidentiality and integrity of transmitted data).

Normal System Operation:

1. The **Sensor Element** continuously samples measurements from the physical environment.
2. A **User** requests the sensor measurements collected during the last hour from the **Server**.
3. The **Server** sends a data collection request to the **Edge Device**.
4. The **Device** performs **Remote Attestation** with the **Server** to prove the integrity of its trusted execution environment.
5. Upon successful attestation, the **Device** and the **Server** establish a secure session and derive a shared session key.
6. The **Device** initiates a **Challenge-Response** protocol with the **Secure Element** using its embedded secret to authenticate the **Sensor Module**.
7. The **Secure Element** successfully proves its identity to the **Device**.
8. The **Sensor** transmits the data to the **Device**.
9. Inside the **Trusted Application (TA)**, the **Device** optionally performs application-specific data processing.
10. The **Trusted Application (TA)** encrypts the resulting data using the session key established with the **Server**, generates an authentication tag (MAC/AEAD), and securely transmits the protected data to the **Server**.
11. The **Server** decrypts the received data, verifies its authenticity and integrity, and returns the requested measurements to the **User**.

Architecture Modeling:

1. Sensor Module

The **Sensor** is responsible for acquiring measurements from the physical environment (e.g., a touchscreen sensor capturing finger movements) and Secure Element responsible for providing a cryptographic identity to the sensor and verify its identity. Conceptually, it consists of a PCB implementing the sensing logic together with an embedded secure element used for sensor authentication.

In our implementation, the sensor module will be **emulated** as a lightweight C program that generates synthetic sensor measurements and stores the embedded secret as a

statically defined global variable. The sensor emulation will execute on the host machine alongside the remaining project components.

2. Device Controller

The **Device Controller** represents the IoT device hosting the sensor. It provides significantly greater computational capabilities than the sensor, manages network communication with the backend server, and supports **ARM TrustZone**, enabling the deployment of Confidential Computing mechanisms.

In our implementation, the Device Controller will be emulated using a **QEMU virtual machine** running an ARM-based IoT platform with **TrustZone** support. Security-sensitive operations will be executed inside **OP-TEE Trusted Applications (TAs)**.

3. Management Server

The **Management Server** represents the backend component responsible for managing the IoT deployment. It collects measurements from Device Controllers, performs application-specific processing when required, and exposes the collected data to users through a well-defined application interface.

In our implementation, the server will be realized as a C application implementing the server logic, communication protocols, and a simple user interface for requesting and displaying sensor measurements.

מה אנחנו נצטרך לעשות:

בתכלס, לייצר את הלוגיקה של רכיב IoT (testbed שממציא נתוני חיישן) שרוצה למסור נתונים לשרת המנהל שלו בצורה מאובטחת.

1. הקמת הסביבה – clone לפרויקט הקיים ויצירת האמולציה של רכיבי IoT (device controller וחיישן).
2. הקמת שרת ב-Azure IoT שיכיל לוגיקה של ניהול רכיבי IoT.
3. מימוש תהליך האטסטציה ב-2 איטרציות כמתואר.
4. מימוש השלמת handshaken ליצירת מפתח סימטרי (בסגנון sigma למשל)
5. מימוש UI מלא למשתמש – הUI ממסך את הקריאות לסביבת הענן, מספק ממשק משתמש נוח לקינפוג המערכת וסטנדרט האבטחה בה, לקרוא נתונים הנאספים מהחיישנים לאחר עיבוד (פלוס מתן ממשק להחלטות על אופן עיבוד הנתונים).
- למשל – המשתמש יוכל, באמצעות לחיצת כפתור מיוחד, לצפות בממוצע משוקלל על פני נתוני החיישנים שנאספו ביממה האחרונה.
6. הדגמת תרחיש מלא של בקשה למסירת מידע מהמשתמש בקצה ברמת האפליקציה, דרך שרת הניהול בענן ועד לאיסוף המידע בצורה מאובטחת ולאחר אימות זהות ואטסטציה מכלל החיישנים.

פונקציונליות:

באמולטור של QEMU

מימוש Edge Device כולל:

1. תקשורת מול השרת
 - a. הצפנה ופענוח של הודעות בסשן ב-TLS
 - b. יצירת מפתחות
 - c. יצירת דוח לאטסטציה מול השרת
 - d. מימוש api למענה לבקשות מידע
2. תקשורת מול ה:sensor module
 - a. Secure element
 - i. C-R
 - b. Sensor element
 - i. בקשת מידע
3. תמיכה בתקשורת מול ה:sensor module
4. תמיכה בקבלת secret אוטומטי על ידי שינוי סטטי / דינמי / קובץ מיועד לכל רכיב IoT יושב ב:refernce path directory שונה לכל אמולציה.

קבצי C נוספים:

1. Sensor
 - a. Api למסירת מידע (סינתטי)
2. Secure Element
 - a. Response ל(HMAC-SHA256) Challenge
 - b. תחזוקה של embedded secret והמפתח התואם באופן סטטי / דינמי / קובץ מיועד לכל רכיב IoT יושב ב:refernce path directory שונה לכל אמולציה.
3. שרת עצמו - user interface, תחזוקת מידע על כל הרכיבים במערכת (regrestion), מימוש הווידוא של דוחות אוטסטציה

הקמה:

רקע:

Cross-Compilation: המטרה שלנו היא לייצר קוד שירוצ על מעבד ARM. כדי לקחת קוד C ולתרגם אותו לשפת מכונה של ARM צריך כלי קימפול מיוחדים (toolchain) שעובדים הכי טוב ב-Linux ומטרתם לייצר את הקומפילציה הנכונה של הקוד ב-C לחומרת ARM.

המניפסט של OP-TEE: קישור - <https://github.com/OP-TEE/manifest> הוא ספר מתכונים שכולל בתוכו את כל ה-repositories שצריך לייבא כדי להרכיב את הפרויקט.

מכיל:

- קרנל של לינוקס (מותאם לQEMU)
- קור מקור של OP-TEE OS
- Toolchain לקומפילציות
- Trusted Firmware של ARM (TF-A).

תאימות: אנחנו צריכים את המניפסט הבא:

https://github.com/OP-TEE/manifest/blob/master/qemu_v8.xml

כדי לעבוד עם QEMU בגרסה מותאמת ל64bit.

דוקומנטציה: <https://optee.readthedocs.io/en/latest>.

Repo: המניפסט של OP-TEE מבוסס על כלי בשם repo של גוגל וכולל Makefiles מורכבים שרצים על Linux. הכלי יבצע קריאה של manifest, יבין אילו repositories נדרשים כדי להרכיב את הפרויקט ויוריד את כולם לתיקייה. בסוף ההורדה עושים make על מנת לייצר את imagen של המערכת ההפעלה השלמה.

Qemu: מתפקד כמעבד וירטואלי של ARM על מערכת הפעלה של Linux. הוירטואליזציה ניתנת כשני טרמינלים שייפתחו במקביל:

1. הטרמינל של העולם הרגיל – מערכת ההפעלה הרגילה (רזה, מיועדת לIoT ולא מאובטחת) של לינוקס. דרכה אנחנו מריצים את הCAs (האפליקציות שמקימות את הTAs ומבקשות הרצות בסביבה מאובטחת מהם).
2. הטרמינל של העולם המאובטח – טרמינל למערכת ההפעלה של OP-TEE. בדרך כלל משמש להדפסת לוגים של המערכת הפעלה המאובטח ושל TAs שרצים בה.

איך כותבים קוד (עושים את הפיתוח של הלוגיקות שאנחנו רוצים בפועל):

הפיתוח יכלול 2 תיקיות:

1. Host – שם כותבים קוד C של הCAs (כמו App.c בSGX)
2. Ta – שם כותבים קוד C של הTAs (כמו Enclave.c בSGX)

קימפול:

המחשב שלנו (זאב) יקבל מערכת build (תיקייה בשם build שנראה לאחר make, דומה לmakefile שצורפו לתיקיות הenclave בSGX) שתדע לקמפל את הTAs, לחתום עליהם דיגיטלית (כדי שOP-TEE תסכים להריץ אותם). את הCAs תקמפל כקובץ הרצה רגיל למערכת הפעלה ללינוקס.

מערכת הbuild יוצרת את imagen הסופי של האמולציה. את קבצי הCAs המקומפלים יחד עם הדרייבר של OP-TEE (רכיב קרנלי שמגשר בין העולמות) היא מכניסה למערכת ההפעלה הרגילה. את הTA המקומפל היא תכניס לOP-TEE המאובטח. נפעיל את הQEMU עם imagen שיצרנו, הדבר הראשון שרץ הוא TF-A (הfirmware המאובטח של ARM). לאחר מכן נטענים שני הטרמינלים כפי שפירטנו –

בטרמינל הראשון נריץ את CA שהוא קורא לדרייבר, מבצע את contact switch לסביבה המאובטחת והמעבד המדומה עובר לעולם המאובטח ויריץ את TA. נראה לוגים בטרמינל 2.

דגש: אם נעשה make אחרי ההורדות של repo ולפני שנכתוב את הקוד בעצמנו מערכות ההפעלה יעלו עם קוד hello world לדוגמה שמגיעה בדיפולט. זה עיקר הרעיון – אנחנו פשוט צריכים לשנות את הלוגיקה בTA וCA בקוד דוגמה שמגיע ומעבר לזה לא נצטרך לגעת במערכת הבUILD והקימפולים. אחרי make נוסף הלוגיקה שתתקמפל תהיה שלנו. ניתן יהיה לראות ולערוך את אותן קבצי דוגמה ולעבוד עליהם בIDE שעובד עם הלינוקס שלנו.

דגש: מכך שאנחנו עובדים על מכונה עם ssh, הרצה רגילה תקרוס כיוון שתנסה לפתוח את הטרמינל על חלון gui – מה שלא קיים בחיבור למכונה מרחוק. לכן נצטרך להריץ את הקוד בתוך ששן של tmux עם פקודה מיוחדת:

```
make run-only NcCns=1
```

run-only לא כזה קשורה פשוט אומרת לOS לא לעשות בדיקות לפני ההרצה (למשל לבדוק שהקבצים לא שונים וצריך לקמפל אותם מחדש) ולדלג ישר להרצה.

NcCns = 1

Nc = Normal Console הקונסול של העולם הלא מאובטח

Cns = Confidential Secure Console הקונסול של העולם המאובטח.

כאשר הערך של הפלאג הזה הוא 1 QEMU לא יפתח שוב חלון גרפי אלא ינתב את הפלט של 2 הטרמינלים לשני פורטים פנימיים של telnet. הסקריפט מזהה שרצים בtmux ומשתמש בממשק שלו כדי לפצל את המסך ולהציג את הפלטים מהפורטים באופן טקסטואלי

הרצה:

שלב ראשון: דרישות קדם –

```
sudo apt install -y adb bc bison\
```

```
build-essential ccache cpio curl flex gawk gcc-arm-linux-gnueabi\
```

```
gcc-arm-linux-gnueabi-hf gcc-aarch64-linux-gnu git libattr1-dev\
```

```
libcap-dev libfdt-dev libftdi-dev libgl2.0-dev libgmp3-dev\
```

```
libhidapi-dev libmpc-dev libncurses5-dev libpixmap-1-dev\
```

```
libssl-dev libtool make mtools netcat-openbsd ninja-build\
```

```
python3-cryptography python3-pip python3-pyelftools\
```

```
python3-serial rsync unzip uuid-dev xxd zlib1g-dev
```

After first compilation try I found out we also need:

```
sudo apt update && sudo apt install -y python3-tomli libgnutls28-dev
```

```
sudo apt-get update && sudo apt-get install -y device-tree-compiler
```

שלב 2: התקנת הכלי repo -

```
mkdir -p ~/.bin  
curl https://storage.googleapis.com/git-repo-downloads/repo > ~/.bin/repo  
chmod a+x ~/.bin/repo  
export PATH=~/.bin:$PATH
```

שלב 3: הפעלת repo על המניפסט של QEMU V8 -

Init repo with the specified manifest:

```
repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
```

Download all repositories based on manifest (might take a while):

```
repo sync -j$(nproc)
```

```
owner@ZEEV:~/natallie/confiden_comp/project$ repo sync -j$(nproc)  
Syncing: 100% (19/19), done in 2m26.349s  
Finalizing sync state...  
repo sync has finished successfully.  
owner@ZEEV:~/natallie/confiden_comp/project$ ls  
SCP-firmware  linux-arm-ffa-user  optee_ftpm  trusted-firmware-a  
build         mbedtls             optee_os    trusted-services  
buildroot     ms-tpm-20-ref      optee_rust  u-boot  
hafnium       optee_client        optee_test  xen  
linux         optee_examples      qemu  
owner@ZEEV:~/natallie/confiden_comp/project$
```

שלב 4: הורדת הקומפיילרים (toolchains) -

בתוך build יש סקריפט מוכן להתקנת toolchains הרלוונטיים (יכול לקחת זמן כי מוריד את הקומפיילרים מהשרתים של ARM):

```
cd build  
make -f toolchain.mk toolchains
```

נשים לב: כחלק מהtoolchains שירדו הסקריפט התקין גם rust:

```
Rust is installed now. Great!

To get started you need Cargo's bin directory
(/home/owner/natallie/confiden_comp/project/build/./toolchains/rust/.cargo/bin)
in your PATH
environment variable. This has not been done automatically.

To configure your current shell, you need to source
the corresponding env file under
/home/owner/natallie/confiden_comp/project/build/./toolchains/rust/.cargo.

This is usually done by running one of the following (note the leading DOT):
.
"/home/owner/natallie/confiden_comp/project/build/./toolchains/rust/.cargo/env"
```

זה נובע מכך שהתשתית תומכת גם בפיתוח rust (לצד C). כדי שbuild יוכל למצוא את הקומפיילר של rust עלינו לטעון אותו כמשתנה סביבה עם:

```
source "/home/owner/natallie/confiden_comp/project/toolchains/rust/.cargo/env"
```

לפני הרצת הmake (ייתכן ואם נדרוש את דוגמאות הקוד שממומשות בRust נוכל לדלג על השלב הזה).

שלב 5: make והרצת דוגמה

Open a tmux session:

```
tmux new -s optee_build
```

Make - first make can take 10 – 20 minutes long, we can safely close the tmux window with ctrl + b then click d and compilation will keep run.

The tee is for convenient logging of completion errors cause tmux not so easy to scroll:

```
make -j$(nproc) 2>&1 | tee compile_full.log
```

```
Image Name:   Linux kernel
Created:      Sun Jun 14 22:37:00 2026
Image Type:   AArch64 Linux Kernel Image (uncompressed)
Data Size:    41413120 Bytes = 40442.50 KiB = 39.49 MiB
Load Address: 42200000
Entry Point:  42200000
ln -sf /home/owner/natallie/confiden_comp/project/build/./out-br/images/rootfs.cpio.gz /home/owner/natallie/confiden_c
/home/owner/natallie/confiden_comp/project/build/./u-boot/tools/mkimage -A arm64 \
    -T ramdisk \
    -C gzip \
    -a 0x45000000 \
    -e 0x45000000 \
    -n "Root file system" \
    -d /home/owner/natallie/confiden_comp/project/build/./out/bin/rootfs.cpio.gz /home/owner/nata
rootfs.cpio.uboot
Image Name:   Root file system
Created:      Sun Jun 14 22:37:00 2026
Image Type:   AArch64 Linux RAMDisk Image (gzip compressed)
Data Size:    22252013 Bytes = 21730.48 KiB = 21.22 MiB
Load Address: 45000000
Entry Point:  45000000
owner@ZEEV:~/natallie/confiden_comp/project/build$
[optee_bui0:~* 1:OPTEE_22040]
```

Run the image:

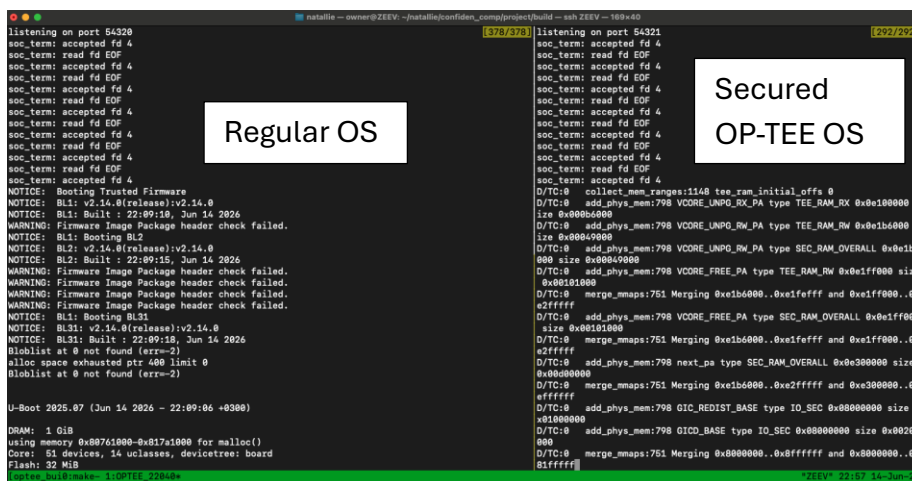
```
make run-only NcCns=1
```

```
owner@ZEEV:~/nattalie/confiden_comp/project/build$ make run-only McNs=1
ln -sf /home/owner/nattalie/confiden_comp/project/build/./out-br/images/rootfs.cpio.gz /home/owner/nattalie/confiden_comp/project/build/./out/bin/

* QEMU is now waiting to start the execution
* Start execution with either a 'c' followed by <enter> in the QEMU console or
* attach a debugger and continue from there.
*
* To run OP-TEE tests, use the xtest command in the 'Normal World' terminal
* Enter 'xtest -h' for help.

* Normal World's terminal has been spawned in OPTEE 24330.
* Secure World's terminal has been spawned in OPTEE 24330.
cd /home/owner/nattalie/confiden_comp/project/build/./out/bin && /home/owner/nattalie/confiden_comp/project/build/./qemu/build/qemu-system-aarch64 -nographic -s
mp 2 -cpu max,sme=on,pauth-impdef=on -d unimp -semihosting-config enable=on,target=native -m 1057 -bios bl1.bin -initrd rootfs.cpio.gz -kernel Image -append 'cons
ole=ttyAMA0,38400 keep_bootcon root=/dev/vda2 ' -machine virt,acpi=off,secure=on,mte=off,gic-version=3,virtualization=false -object rng-random,filename=/dev/uran
dom,id=rng0 -device virtio-rng-pci,rng=rng0,max-bytes=1024,period=1000 -netdev user,id=vmmnic -device virtio-net-device,netdev=vmmnic -s -S -serial tcp:127.0.0.1:
54320 -serial tcp:127.0.0.1:54321
QEMU 10.0.0 monitor - type 'help' for more information
(qemu)
[optee_bui0:make* 1:OPTEE 22040] *ZEEV* 22:52 14-Jun-20
```

Click c (continue) then you will be able to switch windows by clicking the number 1 on the green bar (window 0: make and run window, window 1: our 2 terminals window):



```
listening on port 54320
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
soc_term: accepted fd 4
soc_term: read fd EOF
NOTICE: Booting Trusted Firmware
NOTICE: BL1: v2.14.0(release):v2.14.0
NOTICE: BL1: Built : 22:09:19, Jun 14 2026
WARNING: Firmware Image Package header check failed.
NOTICE: BL1: Booting BL2
NOTICE: BL2: v2.14.0(release):v2.14.0
NOTICE: BL2: Built : 22:09:19, Jun 14 2026
WARNING: Firmware Image Package header check failed.
WARNING: Firmware Image Package header check failed.
WARNING: Firmware Image Package header check failed.
WARNING: Firmware Image Package header check failed.
NOTICE: BL1: Booting BL31
NOTICE: BL31: v2.14.0(release):v2.14.0
NOTICE: BL31: Built : 22:09:19, Jun 14 2026
Bloblist at 0 not found (err=2)
alloc space exhausted ptr 400 limit 0
Bloblist at 0 not found (err=2)

U-Boot 2025.07 (Jun 14 2026 - 22:09:06 +0300)

DRAM: 1 GiB
using memory 0x80761000-0x817a1000 for malloc()
Cores: 51 devices, 14 uclasses, devicetree: board
Flash: 32 MiB

[optee_bui0:make* 1:OPTEE 22040] *ZEEV* 22:57 14-Jun-20
```

Write “root” in left window

```
Set permissions on /dev/tee*: OK
Create/set permissions on /var/lib/tee: OK
Starting tee-supplciant: Using device /dev/teepriv0.
*plugin*: init, version: 0.4.0
OK
Starting network: OK
Starting crond: OK
Starting network (udhcpd): OK

Welcome to Buildroot, type root or test to login
buildroot login: root
# whoami
root
#
[optee_bui0:make* 1:OPTEE 22040] *ZEEV* 22:59 14-Jun-20
```

```
D/TC:0 tee_ta_init_session_with_context:570 Re-open trusted service 3
a2f8978-5dc0-11e8-9c2d-fa7ae81bbebc
D/TC:0 tee_ta_invoke_command:831 Error: ffff000a of 4
D/TC:0 tee_ta_close_session:513 id 1
D/TC:0 tee_ta_close_session:527 Destroy session
D/TC:0 tee_ta_close_session:513 id 2
D/TC:0 tee_ta_close_session:527 Destroy session
D/TC:0 test_wd_callback:49 WD call_count 60, timeout_count 0
D/TC:0 test_wd_callback:49 WD call_count 120, timeout_count 0
D/TC:0 test_wd_callback:49 WD call_count 180, timeout_count 0
D/TC:0 test_wd_callback:49 WD call_count 240, timeout_count 0
D/TC:0 test_wd_callback:49 WD call_count 300, timeout_count 0
D/TC:0 test_wd_callback:49 WD call_count 360, timeout_count 0
```

Now run HelloWorld example:

```
Welcome to Buildroot, type root or test to login
buildroot login: root
# whoami
root
# optee_example_hello_world
Invoking TA to increment 42
TA incremented value to 43
#
[optee_bui0:make* 1:OPTEE 22040]
```

Final outputs:

```
[ 2.047687] Key type encrypted registered
[ 2.06246] input: gpio-keys as /devices/platform/gpio-keys/input/input0
[ 2.062279] clk: Disabling unused clocks
[ 2.062581] PM: genpd: Disabling unused power domains
[ 2.395971] Freeing unused kernel memory: 3264K
[ 2.398123] Run /init as init process
Saving 256 bits of creditable seed for next boot
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Set permissions on /dev/tee: OK
Create/set permissions on /var/lib/tee: OK
Starting tee-supplcatt: Using device /dev/teepriv0.
*plugins: init, version: 0.4.0
OK
Starting network: OK
Starting crond: OK
Starting network (udhcpd): OK

Welcome to Buildroot, type root or test to login
buildroot login: root
# whoami
root
# optee_example_hello_world
Invoking TA to increment 42
TA incremented value to 43
# ||

D/TC: ? 0 tee_ta_init_pseudo_ta_session:303 Lookup pseudo TA 8aaaf200-2450-11e4-abe2-0002a5d5c51b
D/TC: ? 0 ldelf_load_ldelf:110 ldelf load address 0x40007000
D/LD: ldelf:159 Loading TS 8aaaf200-2450-11e4-abe2-0002a5d5c51b
D/TC: ? 0 ldelf_syscall_open_bin:163 Lookup user TA ELF 8aaaf200-2450-11e4-abe2-0002a5d5c51b (early TA)
D/TC: ? 0 ldelf_syscall_open_bin:167 res=0xffff0000
D/TC: ? 0 ldelf_syscall_open_bin:163 Lookup user TA ELF 8aaaf200-2450-11e4-abe2-0002a5d5c51b (Secure Storage TA)
D/TC: ? 0 ldelf_syscall_open_bin:167 res=0xffff0000
D/TC: ? 0 ldelf_syscall_open_bin:163 Lookup user TA ELF 8aaaf200-2450-11e4-abe2-0002a5d5c51b (REE)
D/TC: ? 0 ldelf_syscall_open_bin:167 res=0
D/LD: ldelf:207 ELF (8aaaf200-2450-11e4-abe2-0002a5d5c51b) at 0x400440
00
D/TA: TA_CreateEntryPoint:18 has been called
D/TA: __GP11_TA_OpenSessionEntryPoint:47 has been called
I/TA: Hello World!
D/TA: inc_value:78 has been called
I/TA: Got value: 42 from MW
I/TA: Increase value to: 43
D/TC: ? 0 tee_ta_close_session:513 id 4
D/TC: ? 0 tee_ta_close_session:527 Destroy session
I/TA: Goodbye!
D/TA: TA_DestroyEntryPoint:29 has been called
D/TC: ? 0 destroy_context:326 Destroy TA ctx (0xc3f3750)

*ZEEV* 23:00 14-Jun-20
```

מה רואים פה?

הריצה באטום tmu פתחה שני חלונות של 2 הטרמינלים שלנו. הטרמינל הימני הוא הלוגים שהקרנל של OP-TEE מעדכן על המתרחש ב-OS המאובטח. בצד השמאלי זהו הטרמינל של מערכת ההפעלה הלא מאובטחת הרגילה (מתפקדת כ-CL רגיל של לינוקס). ב-`hello world` התוכנית מבקשת מה-`TA` לעדכן ערך שניתן בקלט מ-42 ל-43 (`increment`).

ה-`prefix` בלוגים של ה-`OP-TEE` מסודרים באופן הבא:

Log level / the component printed it

Log levels:

- `D (Debug)` - הודעות דיבוג פנימיות. הן נועדו לעזור למפתח להבין את זרימת התוכנית (למשל: "פונקציה `X` נקראה" או "משתנה `Y` שווה ל-42"). בסביבת ייצור (`Production`) אמיתית בשוק, בדרך כלל מכבים אותן כדי לחסוך ביצועים.
- `I (Info)` - הודעות מידע כלליות על אירועים חשובים אך תקינים (למשל: "מערכת עלתה", "האפליקציה נסגרה").
- `W (Warning)` - אזהרה. משהו לא התנהג כמצופה, אבל המערכת הצליחה להתגבר עליו ולא קרסה.
- `E (Error)` - שגיאה חמורה. פעולה נכשלה לחלוטין (למשל: "אין מספיק זיכרון מוגן" או "החתימה הדיגיטלית שגויה").

הרכיב שמדפיס:

`TA (Trusted Application)` - הודעות שמגיעות מהמפתח של האפליקציה המאובטחת. הקוד שנכתוב בתוך ה-`TA`). אפשר להשתמש בפקודות כמו `DMSG` או `IMSG` כדי להדפיס למסך מתוך ה-`TA`.

`TC (Trusted Core)` - הודעות שמגיעות מהלב של ה-`kernel` המאובטח. הקוד שלנו לא שלט בזה, ה-`kernel` מדווח שהוא מנהל את הסשנים, מקצה זיכרון או עובר בין מצבים.

`LD (Internal Linker/Loader)` - הודעות מרכיב המערכת שאחראי לטעון את הקבצים לזיכרון המאובטח. כשמפעילים אפליקציה, ה-`LD` מתעורר, לוקח את קובץ ה-`TA`, בודק שהחתימה שלו תקינה, וממקם אותו בזיכרון.

ה-`flow` של התוכנית לדוגמה:

בצד הTA:

```
00
D/TA: TA_CreateEntryPoint:18 has been called
D/TA: __GP11_TA_OpenSessionEntryPoint:47 has been called
I/TA: Hello World!
D/TA: inc_value:78 has been called
I/TA: Got value: 42 from NW
I/TA: Increase value to: 43
D/TC:? 0 tee_ta_close_session:513 id 4
D/TC:? 0 tee_ta_close_session:527 Destroy session
I/TA: Goodbye!
D/TA: TA_DestroyEntryPoint:29 has been called
D/TC:? 0 destroy_context:326 Destroy TA ctx (0xcc3f3750)
D/TC:0 test_wd_callback:49 WD call_count 420, timeout_count 0
D/TC:0 test_wd_callback:49 WD call_count 480, timeout_count 0
D/TC:0 test_wd_callback:49 WD call_count 540, timeout_count 0
```

בצד הCA:

```
Welcome to Buildroot, type root or test to login
buildroot login: root
# whoami
root
# optee_example_hello_world
Invoking TA to increment 42
TA incremented value to 43
#
```

יצירת הTA:

- D/TA: TA_CreateEntryPoint:18 has been called – הפונקציה נקראת בתחילת חיי הTA במטרתה להקצות את מרחב הזיכרון שלו במערכת המאובטחת.
- D/TA: __GP11_TA_OpenSessionEntryPoint:47 has been called – אפליקציית הCA מהOS הרגיל מבקשת לפתוח סשן מול הTA. במידה ויש לCA אישור להריץ את הTA התקשורת מתחילה.
- I/TA: Hello World – הודעה מתוך הTA (לאחר שהוא כבר טעון ורץ במרחב זיכרון המבודד) שאומרת שהתוכנית נטענה ומתחילה לרוץ.

ביצוע המשימה הסודית:

- D/TA: inc_value:78 has been called – מספר השורה בקובץ הC של הTE הוא 78 שם יש קריאה לפונקציה inc_value.
- I/TA: Got value: 42 from NW - היא מדפיסה הודעה על כך שקיבלה את המספר 42 מהעולם הרגיל NormalWorld.
- I/TA: Increase value to: 43 - היא ביצעה את החישוב המתמטי בתוך האזור המאובטח (כלומר כאן היה Context Switch לsecure world).

סגירת הסשן:

- D/TC:? 0 tee_ta_close_session:527 Destroy session – אפליקציית הCA קיבלה תשובה וסיימה וכעת מבקשת לסגור את הסשן.
- I/TA: Goodbye! - היא החזירה את התשובה ללינוקס ומודיעה על סגירת הסשן
- D/TA: TA_DestroyEntryPoint:29 has been called – מכיוון שאין יותר handles משתמשים בinstances של הTA הקרנל הורס את מרחב הזיכרון של הTA ומנקה אותו (פונקציית destructorn בשורה 29).

- Destroy TA ctx – הודעה כי הסשן נהרס ונוקה בצורה מאובטחת.

כדי לסגור את הסשן על זאב בצורה מסודרת:

Write poweroff inside the left terminal:

```
# poweroff
# Stopping network (udhcpd): OK
Stopping crond: stopped /usr/sbin/crond (pid 122)
OK
Stopping network: OK
Stopping tee-suplicant: FAIL
Stopping klogd: stopped /sbin/klogd (pid 88)
OK
Stopping syslogd: stopped /sbin/syslogd (pid 84)
OK
Seeding 256 bits and crediting
Saving 256 bits of creditable seed for next boot
umount: devtmpfs busy - remounted read-only
The system is going down NOW!
Sent SIGTERM to all processes
Sent SIGKILL to all processes
Requesting system poweroff
[ 3318.238710] Flash device refused suspend due to active operation (state 20)
[ 3318.241145] reboot: Power down
ERROR: QEMU System Power off: with GPIO.
soc_term: read fd EOF
[Optee build:tmux]- 1:OPTEE 22040*
```

Kill tmux window:

Press ctrl + b then click &:

```
kill-window OPTEE_22040? (y/n) press y
```

In the main tmux window (window 0) click ctrl c to finish the invocation of the session.

You can kill this tmux session in the same way if you want or keep it for later (press ctrl + b then click d for exiting the tmux window).

מבנה התיקיות ותוכן הקבצים:

תיקיית הproject שבה התקנו את build יחד עם כל הקוד של השתית מכילה מספר תיקיות עיקריות:

```
owner@ZEEV:~/nataillie/confiden_comp/project$ ls
SCP-firmware  buildroot  linux      mbedtls      optee_client
optee_ftpm    optee_rust out        qemu         trusted-firmware-a  u-boot
build          hafnium    linux-arm-ffa-user  ms-tpm-20-ref  optee_examples
optee_os       optee_test out-br     toolchains   trusted-services    xen
owner@ZEEV:~/nataillie/confiden_comp/project$
```

- optee_examples – תיקיית דוגמאות הקוד הרשמית של הפרויקט (כולל hello_world שהרצנו). את המבנה קבצים שם נבין, נעתיק ונערוך לפי המטרות שלנו.
- optee_os – הקרנל של העולם המאובטח. לא נוגעים פה.
- optee_client – ספריות C של העולם הרגיל, מכילה את APIs שמאפשרת לנו לדבר עם העולם המאובטח. קוד הCAN יתבסס על הספריות הללו.
- optee_test – סט בדיקות לפיתוח של החומרה (לא נוגעים).

- Mbedtls – סיפריית הקריפטוגרפיה, מותאמת ל-ARM. נמצאת בשימוש ב-OP-TEE כדי לבצע פעולות קריפטוגרפיות.
- optee_ftpm – מימוש תוכנית של רכיב TPM (רכיב הצפנה חומרתי מאובטח) בתוך TrustZone.
- optee_rust – תאימות לפיתוח ב-rust במקום ב-C.
- Build – מערכת הקומפילציה שלנו
- Out – בינאריים סופיים.

כדי לפתוח בנוחות את הפרויקט ב-Clion (למעוניינים) עם ניווט נוח בין הקבצים ואינדוקס קליל של הקבצים:

1. הוספה של הניתוב ל-optee_client optee_osi Cmake של תיקיית הדוגמאות:

```
GNU nano 6.2 CMakeLists.txt
cmake_minimum_required(VERSION 3.5)
project(optee_examples C)

# https://cmake.org/Wiki/CMake_Useful_Variables
set(CMAKE_TOOLCHAIN_FILE CMakeToolchain.txt)

include(GNUInstallDirs)

add_compile_options(-Wall)
#add_compile_options(
#
#    -Wall -Wbad-function-cast -Wcast-align
#    -Werror-implicit-function-declaration -Wextra
#    -Wfloat-equal -Wformat-nonliteral -Wformat-security
#    -Wformat=2 -Winit-self -Wmissing-declarations
#    -Wmissing-format-attribute -Wmissing-include-dirs
#    -Wmissing-noreturn -Wmissing-prototypes -Wnested-externs
#    -Wpointer-arith -Wshadow -Wstrict-prototypes
#    -Wswitch-default -Wunsafe-loop-optimizations
#    -Wwrite-strings -Werror -fPIC
#)

include_directories(
    /home/owner/natalie/confiden_comp/project/optee_client/libteec/include
    /home/owner/natalie/confiden_comp/project/optee_os/out/arm/export-ta_arm64/include
)

find_program(CCACHE_FOUND ccache)
if(CCACHE_FOUND)
    set_property(GLOBAL PROPERTY RULE_LAUNCH_COMPILE ccache)
    set_property(GLOBAL PROPERTY RULE_LAUNCH_LINK ccache)
endif(CCACHE_FOUND)

file(GLOB dirs *)
foreach(dir ${dirs})
    if(EXISTS ${dir}/CMakeLists.txt)
        add_subdirectory(${dir})
    endif()
endforeach()
```

נוסיף את הניתוב במיקום המוגדר למעלה.

2. נפתח את הסשן ב-Clion על זאב בתיקייה של optee_examples.



זה יתחיל תהליך אינדוקס ראשוני של הקבצים (אמור לקחת גג כמה דקות).

- נפתח את קובץ Cmake (של כלל התיקייה – לכל פרויקט / תת תיקייה יש Cmake משלה, אנחנו עובדים מול Cmake של תיקיית הדוגמאות) שם יופיע bar של Fix בקצה הימני של הקובץ שיציע לנו לקמפל את הפרויקט כ-cmake – נבחר אותו ונחכה שהקימפול יסתיים.
- בקובץ Cmake של hello_world יופיע bar עם אייקון של כובע קוסמים על מנת לרענן את הקימפול – נלחץ עליו.
- לאחר מכן Clion יכיר ויצליח לנווט לכל רכיבי הקוד שמוזכרים בקבצים עליהם נעבוד:

```
1 // SPDX-License-Identifier: BSD-2-Clause
2 /*
3  * Copyright (c) 2016, Linaro Limited
4  * All rights reserved.
5  */
6
7 #include <tee_internal_api.h>
8 #include <tee_internal_api_extensions.h>
9
10 #include <hello_world_ta.h>
11
12 /*
13  * Called when the instance of the TA is created. This is the first call in
14  * the TA.
15  */
16 TEE_Result TA_CreateEntryPoint(void)
```

דגש: התהליך נועד נטו לפיתוח נוח בClion – cmake עוטף את הפרויקט, Clion יודע לעבוד מול CMakeList וולנווט באמצעותו. כדי לקמפל את הקוד ולהריץ אותו בפועל צריך לעבוד עם make (לא cmake) על בסיס Makefiles המתאימים שאת הניהול והאינטגרציה בניהם build מכיר ויודע לעבוד מולם. כלומר הקימפול וההרצה נעשים כמתואר למעלה עם הטרמינל וtmux!

דגש: ייתכן וניווטים נוספים יהיו חסרים, כדי להתגבר על הקבצים החסרים צריך לברר את מיקום קובץ ה.h. שהקוד מחפש בתיקיית root של project ולהוסיף אותו לinclude_directories בCMakeList של תיקיית הדוגמאות.